C++ Programming Textbook Notes

Module: Strings in C++ | Legacy C-Strings, Modern std::string & Input Mechanics

Author: Gaurav Kandpal **Platform:** CS-Simplified **Level:** Beginner to Advanced

1. INTRODUCTION TO STRINGS IN C++

Conceptual Explanation

In C++, a **String** represents a sequential sequence of characters used to store and manipulate textual data (e.g., names, sentences, messages).

Unlike C-style character arrays (`char[]`) which require manual buffer sizing and pointer arithmetic, modern C++ provides `string` as part of the Standard Template Library (STL). Objects of the `string` class handle memory allocation and dynamic resizing automatically while exposing rich built-in functions for searching, concatenation, and extraction.

Tabular Paradigm Comparison

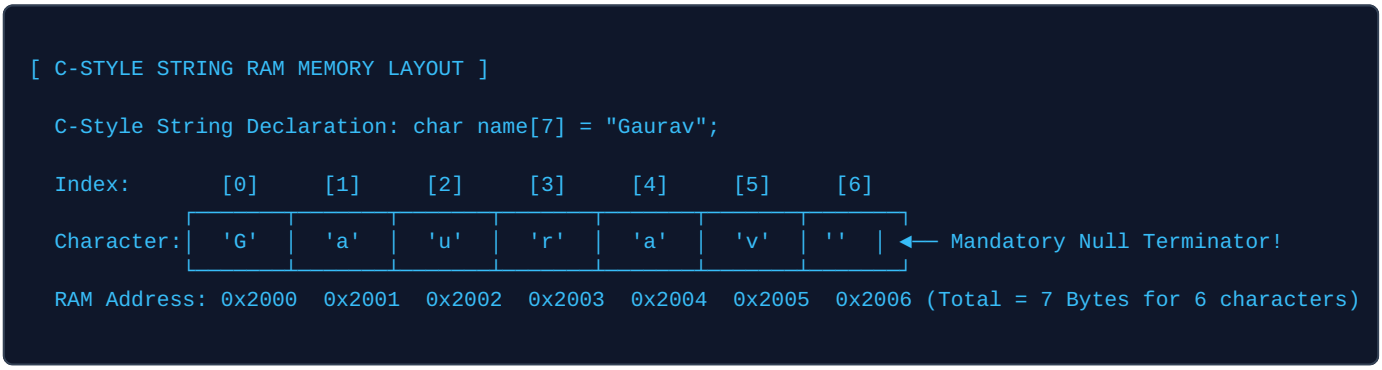
Feature	C-Style Strings (`char[]`)	Modern C++ `string` (`<string>`)
Memory Location	Stack array or Read-only Data Segment.	Heap buffer (SSO optimized on Stack for short strings).
Resizing Ability	Fixed, static buffer size set at declaration.	Dynamic automatic expansion during runtime.
Null Termination	Mandatory manual `''` terminator byte.	Maintained automatically by internal class handle.
Copy Overhead	Manual `strcpy`; expensive / error-prone.	Deep copy (O(N) operation) or Move semantics (O(1)).

2. C-STYLE STRINGS (`CHAR[]` & THE NULL-TERMINATOR)

Conceptual Explanation & Memory Layout

A **C-style string** is stored as a 1D array of characters in contiguous memory bytes. To signal where the textual string ends, C appends an invisible ASCII value `0` byte known as the **null terminator** (`''`).

Without `''`, functions like `strlen()` or `cout` do not know where the string stops. They continue reading past array bounds into adjacent RAM addresses, causing **garbage character output** or **segmentation fault memory crashes**.



Legacy C-String Code Demonstration

```cpp
// Legacy C-Style Character Array Demonstration
#include <iostream>
#include <cstring> // Header for legacy c-string utility functions
using namespace std;

int main() {
    // Explicit array initialization requires 1 extra byte for ''
    char str1[] = "Hello";   // Automatically sized to 6 bytes in RAM
    char str2[20] = "World"; // Sized to 20 bytes buffer

    // Legacy C functions
    cout << "Length of str1 (strlen): " << strlen(str1) << endl; // Outputs 5
    cout << "Byte Size of str1 (sizeof): " << sizeof(str1) << " bytes" << endl; // Outputs 6

    // Concatenation via strcat
    strcat(str2, " C++"); // Danger: str2 buffer must be large enough!
    cout << "Concatenated str2: " << str2 << endl;

    return 0;
}
```

Buffer Overflow Hazard in C-Strings

Legacy C-string functions like `strcpy()` and `strcat()` perform **zero buffer boundary checking**. Writing a 15-character string into a 10-byte character array corrupts adjacent stack RAM, introducing severe security vulnerabilities!

3. MODERN C++ `STD::STRING` (`<STRING>`)

Conceptual Explanation

Modern C++ provides `std::string` as part of the Standard Template Library (STL). A `std::string` object encapsulates three essential fields internally:

- **Pointer (`char*`):** Address pointing to the dynamic character buffer on the Heap.
- **Size (`size_t`):** Exact count of valid stored characters.
- **Capacity (`size_t`):** Total memory allocated before requiring reallocation.

Syntax

The string container is defined as `std::string` class inside the `<string>` header file.

```
string string_name ;
```

Key Member Functions Reference Table

Category	Member Function	Description
Access	`str[i]` / `str.at(i)`	Direct access; `.at(i)` performs safe bounds checking throwing `std::out_of_range`.
Sizing	`str.length()` / `str.size()`	Returns total character count.
Modification	`str += "text"` / `str.append()`	Concatenates additional characters automatically reallocating heap memory.
Search	`str.find("pattern")`	Returns starting index or `std::string::npos` if search fails.
Extraction	`str.substr(pos, count)`	Extracts a new sub-string starting at pos with count characters.

Comprehensive `std::string` Operations Code Example

```cpp
// Modern C++ string Operations Demonstration
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

int main() {
    // Initialization
    string firstName = "Gaurav";
    string lastName = "Sharma";

    // Concatenation using + operator
    string fullName = firstName + " " + lastName;
    cout << "Full Name: " << fullName << endl;

    // Modification & Search
    fullName.append(" (Developer)");
    size_t pos = fullName.find("Sharma");

    if (pos != string::npos) {
        cout << "Found 'Sharma' at index: " << pos << endl;
    }

    // Substring Extraction
    string sub = fullName.substr(0, 6); // Extracts 6 chars starting at index 0
    cout << "Extracted Substring: " << sub << endl;

    // Safe Access with .at()
    try {
        cout << "Char at index 2: " << fullName.at(2) << endl;
    } catch (const out_of_range& e) {
        cout << "Bounds check failed: " << e.what() << endl;
    }

    return 0;
}
```

4. STRING INPUT MECHANICS (`CIN >>` VS. `GETLINE`)

Conceptual Explanation

Reading strings from standard input requires understanding whitespace delimiter rules:

- `cin >> str;`: Reads single words. Stops reading immediately at the first whitespace (space, tab, or newline character).
- `getline(cin, str);`: Reads complete sentences including spaces until the enter key (`'\n'`) is pressed.

Text-Based Flowchart: Input Stream Whitespace Trap

```
Scenario: Executing 'cin >> age' followed by 'getline(cin, fullName)'

User Types: "20\n"
  ──► cin >> age Reads 20 ──► '\n' Remains stranded in Input Buffer!
                                                          │
                                                          ▼
                        getline Execution Reads Stranded '\n'
                                                          │
                                                          ▼
                        Result: 'fullName' is assigned empty string! (Skipped Prompt)
```

Input Handling Code Example

```cpp
// Fixing the Stranded Buffer Trap using cin.ignore()
#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main() {
    int age;
    string fullName;

    cout << "Enter Age: ";
    cin >> age;

    // CRITICAL FIX: Flush trailing '\n' from stream buffer before calling getline
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter Full Name: ";
    getline(cin, fullName); // Safely reads full sentence with spaces

    cout << "Age: " << age << " | Name: " << fullName << endl;

    return 0;
}
```

Important Points & Gotchas

⚠️ **Stranded Buffer Trap:** Mixing `cin >>` and `getline()` without calling `cin.ignore()` causes `getline` to instantly consume the leftover newline byte, skipping user input!